

AdaCoM

Learning Agent-Compatible Context Management for Long-Horizon Tasks - 深度研究与工程 Know-how

核心转变：Context Management 不再是 Agent 的附属能力，而是可独立训练、可迁移、可适配闭源 Agent 的 Policy Layer

Long-Horizon Context Management：从压缩启发式到独立可学习 Policy



图 1 长程 Agent Context Management 的方法演进

研究日期：2026-08-13 | 核心来源：AdaCoM arXiv:2605.30785 v1 + 相关 2025-2026 Context Management 工作

目录

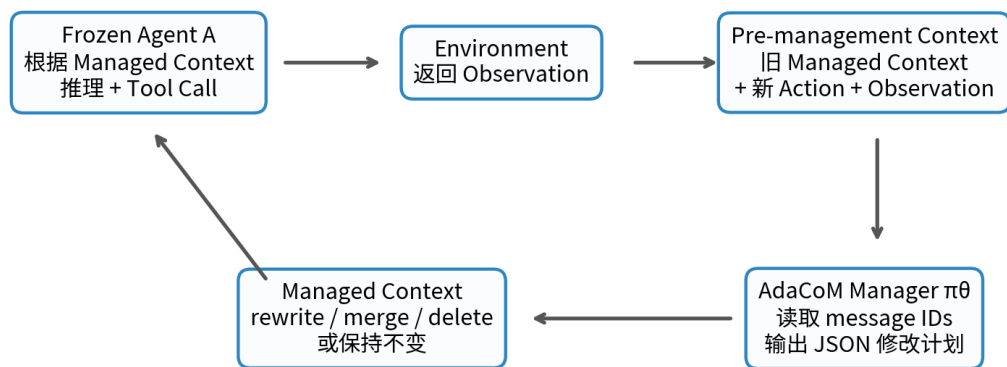
1. 一页结论：AdaCoM 到底新在哪里
 2. Know-why：为什么“让 Agent 自己管理 Context”不是通用解
 3. 系统架构：Frozen Agent + External Context Manager
 4. Flexible Modification Action Space：通用 Message Edit IR
 5. RL：如何训练一个“懂 Agent”的 Context Policy
 6. Process Reward 与 Two-level Advantage
 7. 实验设计：Web Search + Deep Research
 8. 主要实验结果
 9. Fidelity-Reliability Trade-off
 10. Cross-Agent Transfer：Context Manager 能否复用
 11. 数据构造与训练工程
 12. 与 ReSum / LongSeeker / ACM / ACE / PACE 的关系
 13. 生产级 Deep Research 的可迁移 Know-how
 14. 局限、风险与下一代架构
 15. 结论
- 参考资料

1. 一页结论：AdaCoM 到底新在哪里

AdaCoM 的研究对象不是“怎么让上下文更短”，而是“针对一个给定 Agent，什么样的上下文表示能让它持续可靠地完成长程任务”。它将 Context Management 从主 Agent 中解耦：主 Agent A 完全冻结，一个较小的外部 Context Manager $\pi\theta$ 在每次 action-observation 之后重写主 Agent 下一步将看到的 message sequence。

- Architectural decoupling: 只训练 Context Manager，不修改主 Agent，因此理论上可以适配闭源模型/API Agent。
- Operation-level flexibility: 不是预定义“summarize”工具，而是对任意 message IDs 进行 rewrite / merge / delete / no-change。
- End-to-end RL: Context Manager 的 edit policy 由最终任务 reward 与中间 process reward 联合训练。
- Agent compatibility: 不同 Agent 学到不同的上下文长度与记忆风格；“统一摘要模板”并不是最优策略。
- Transfer: 训练好的 Manager 可迁移到未参与训练的 Agent，尤其当 source/target Agent 的基础能力和 reasoning style 接近时。

AdaCoM Runtime：训练 Context Manager，不训练 Agent



Task Reward / Process Reward 只更新 $\pi\theta$ ；Frozen Agent 与工具环境保持不变

图2 AdaCoM：冻结主 Agent，只优化外部 Context Policy

2. Know-why：为什么“让 Agent 自己管理 Context”不是通用解

2.1 现有方法的隐含假设

ReSum、MEM1、IterResearch、MemAct、Context-ReAct 等路线虽然实现方式不同，但许多方案都隐含一个假设：主 Agent 能理解新的 context pattern 或新的 memory/context tool protocol。问题是，模型未必在训练中见过这种协议。

- 如果 Agent 被要求在同一 generation 里“总结过去 + 决定下一步”，它必须同时承担 reasoning policy 与 context policy。
- 如果添加 prune/compress 工具，Agent 还必须学会何时调用、选择哪些历史、摘要到什么粒度。
- 对闭源 Agent，无法轻易追加 SFT/RL 来适应这些新的上下文模式。

2.2 Fixed Summarization 的 one-size-fits-all 问题

不同 Agent 对长上下文的耐受程度、对原始细节的利用能力、对摘要文本风格的偏好都不同。因此“每 N 步总结一次”或“到阈值就统一摘要”很可能把弱 Agent 留在过长 context 中，也可能过度压缩强 Agent 能利用的细粒度证据。

2.3 AdaCoM 的核心假设

Context Management 应被看成一个独立控制问题：给定当前任务、历史 action、tool observations 和主 Agent 的行为特征，学习一个变换 $T\theta(c)$ ，使下游 frozen Agent 的最终 task reward 最大。

3. 系统架构：Frozen Agent + External Context Manager

标准 ReAct 在 turn t 维护 $c_t=(q,a_1,o_1,...,a_t,o_t)$ ，下一步直接用全部历史生成 a_{t+1} 。AdaCoM 引入 managed context c_t : Agent 先基于 c_{t-1} 行动，得到新 observation 后形成 pre-management context c_t ；随后 Manager 根据 c_t 输出结构化修改动作 m_t ，得到 c_t ，再交给 Agent。

对象	是否训练	职责	工程边界
Frozen Agent A	否	推理、决定 tool call、输出答案	可以是闭源 API
Context Manager π_θ	是	重写/合并/删除/保留 message sequence	Qwen3-4B-Instruct 初始化
Environment / Tools	否	search / get_document / Wikipedia MCP	提供外部 observation
Reward / Judge	训练时	评估最终任务结果与部分中间行为	不进入生产主路径也可

最关键的系统设计是“控制面和执行面分离”：Agent 是任务执行策略；AdaCoM 是 Working Context 控制策略。二者可独立升级、独立路由、独立评测。

4. Flexible Modification Action Space：通用 Message Edit IR

Flexible Modification Action Space：不是五个固定工具，而是一种通用 Message Edit IR

字段	语义	可实现的 Context 操作	Know-how
ids	选中一个/多个 message IDs	局部编辑 / 跨消息 merge	显式 addressing，避免整段重写
role	SYSTEM / USER / ASSISTANT	改变结果 message 的角色	可构造稳定 task-state message
justification	编辑理由；不会交给主 Agent	训练 manager 的 decision rationale 理由用于学习，不污染工作上下文	
new_content	新文本；空字符串表示删除	rewrite / condense / merge / delete etc	统一 edit language，不绑定 summary primitive
[]	空 operation list	no_change	保真本身也是一种策略

图 3 AdaCoM 的动作空间本质上是 Message Edit Intermediate Representation

论文没有把 Context Manager 限制成固定 summarize / delete 工具，而是让它输出 JSON modification list。每个 operation 指向一组 message IDs，并指定 role、justification、new_content。

4.1 为什么这是一个强设计

- 统一性：rewrite、摘要、merge、delete 都能由同一个 IR 表示。
- Addressability：基于 message IDs 做局部修改，避免每次把整个历史重新生成。
- Role-aware：可以生成 system/user/assistant 类型的状态消息，形成主 Agent 更容易利用的 working-memory style。
- No-change 是一等动作：Context Policy 可以选择不动，而不是被迫压缩。

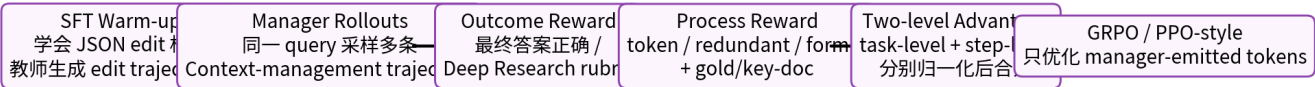
4.2 论文中出现的实际行为

成功轨迹里，Manager 经常在用用户任务后维护一个 compact state message，持续记录：任务要求、未满足约束、有用证据、当前 leads、已拒绝候选、无效 query。作者强调这不是显式 prompt 指定的模板，而是训练后自行涌现出的组织方式。

在 MCP-Bench-Wiki 中，对超长 get_article 结果还增加 extract 行为：先保留 task-relevant sentences，再进入主 Agent context，避免单个长网页直接撑爆 32K manager budget。

5. RL：如何训练一个“懂 Agent”的 Context Policy

AdaCoM 的 RL：Credit Assignment 的对象是“上下文编辑动作”



$$A_{i,t} = \text{normalized task advantage} + \alpha \times \text{normalized process advantage} ; \text{随后再次归一化}$$

图 4 AdaCoM 训练链路：最终任务信号 + Context-specific process reward

AdaCoM 先用 SFT 让 Qwen3-4B-Instruct 学会 edit schema，再用 GRPO 风格的在线 RL。关键点是 rollout 中主 Agent 始终冻结，因此 Manager 必须通过“改变 Agent 看见什么”间接提高最终 reward。

训练要素	论文设置	Know-how
Manager backbone	Qwen3-4B-Instruct	Context Policy 可以显著小于主 Agent
SFT teacher	BrowseComp: GPT-5; MCP-Wiki: Claude Opus 4.6	强教师用于生成高质量 edit demonstrations
Group size	G=8	同一 query 下比较多条 manager trajectories
Manager input	32,768 tokens	主动限制控制器的工作预算
Manager output	4,096 tokens	允许结构化重写，但避免无限生成
Max iterations	35	长程但有明确 horizon

这形成一个很有价值的部署思想：不需要“把每个生产 Agent 都重新 RL 一遍”，可以把训练预算集中到一个 Context Control Model。

6. Process Reward 与 Two-level Advantage

仅靠最终答案 reward，Manager 很难知道第 8 步的压缩是否导致第 25 步失败。因此 AdaCoM 加入 rule-based process reward，并把 outcome reward 与 process reward 分别标准化后合并。

Process signal	权重/规则	它在惩罚什么
Token penalty	-0.8	Manager 产出超过 context budget 的 working context
Redundant-action penalty	-0.4	Agent 连续两次发出同 tool + 同参数
Format penalty	-0.5	JSON 无法解析、ID 不存在、字段缺失
Gold-doc-found	+0.6 (BrowseComp)	编辑后的 context 导致下一步找到 gold document
Key-doc-found	+0.3 (BrowseComp)	编辑后的 context 导致下一步找到 key document

训练时先在 rollout group 内把 terminal outcome reward 变成 task-level normalized advantage，再把所有 manager step 的 process reward 变成 step-level normalized advantage，二者以 $\alpha=0.1$ 合并并再次归一化。这样做的 know-why 是避免两种 reward 数值尺度不同而让训练不稳定。

消融也验证了 process reward 的必要性：DeepSeek 的 mean@3 从 26.19 降到 24.38，Qwen 从 36.67 降到 33.33。

7. 实验设计：Web Search + Deep Research

7.1 BrowseComp-Plus

使用受控、可验证 corpus 的 BrowseComp-Plus。训练/测试为 680/150；工具只有 search(query, top_k) 和 get_document(doc_id)，因此实验重点更接近“搜索策略 + Context Policy”，而不是网页解析噪声。最大 35 iterations，GPT-4o 作为 correctness judge。

7.2 MCP-Bench-Wiki

Deep Research 侧构造 Wikipedia MCP benchmark，提供 9 个 Wikipedia MCP 工具，包括 search、全文读取、摘要、section、key facts、links 等。最终评价重点选 TF (task fulfillment)、IG (information grounding)、PE (parallelism & efficiency)，Overall = 0.4 TF + 0.4 IG + 0.2 PE。

任务	主要压力	为什么适合评测 Context Manager
BrowseComp-Plus	多约束、多次搜索、候选排除	容易暴露 constraint forgetting / repeated search
MCP-Bench-Wiki	多工具、多文档、长报告式答案	容易暴露长 observation、evidence grounding 与 tool efficiency 问题

8. 主要实验结果

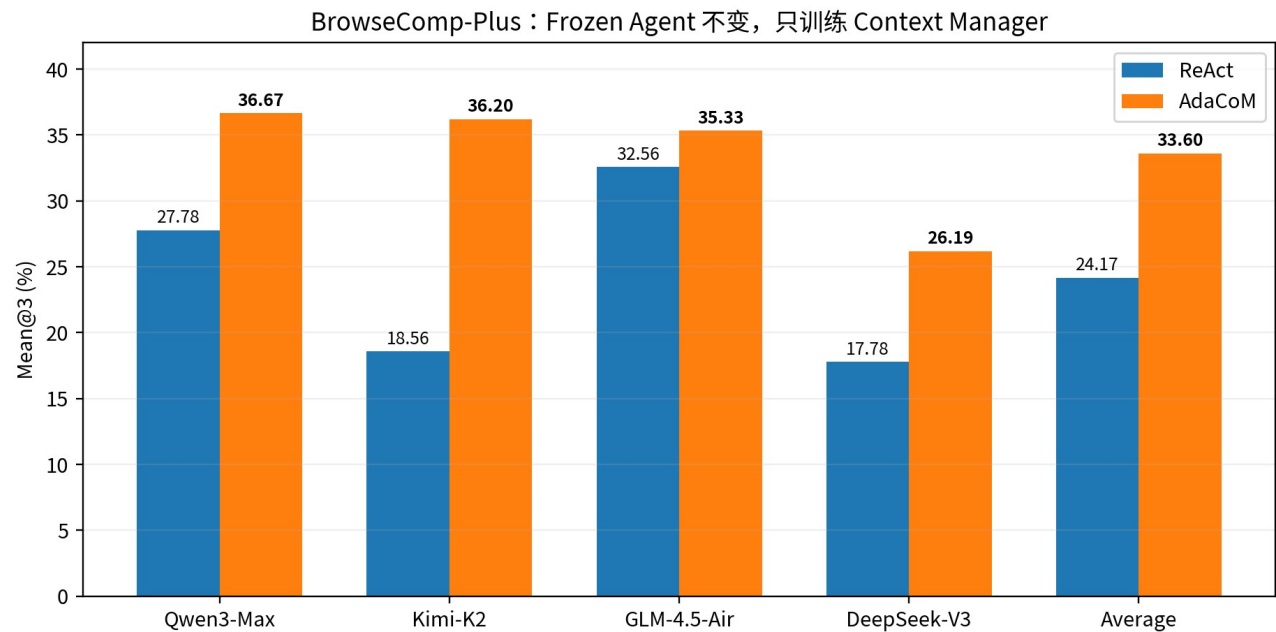


图5 AdaCoM 在 4 个不同 Frozen Agent 上均提升 BrowseComp-Plus Mean@3

BrowseComp-Plus 的核心证据是“一致性”：AdaCoM 对 Qwen、Kimi、GLM、DeepSeek 四种 Agent 都高于它们自己的 ReAct baseline，平均从 24.17 提升到 33.60，相对平均增益 39.0%。未训练的 AdaCoM 反而平均下降，说明“有一个外部编辑器”本身不够，必须学习 agent-compatible policy。

Agent	ReAct	AdaCoM	相对变化
Qwen3-Max	27.78	36.67	+32.0%
Kimi-K2-Instruct	18.56	36.20	+95.0%
GLM-4.5-Air	32.56	35.33	+8.5%
DeepSeek-V3	17.78	26.19	+47.3%
Average	24.17	33.60	+39.0%

在 MCP-Bench-Wiki，Kimi 从 55.05 到 60.01（+9.0%），DeepSeek 从 47.51 到 58.09（+22.3%），表明外部 Context Policy 的收益并不限于短答案 web search，也扩展到 tool-use-intensive deep research。

9. Fidelity-Reliability Trade-off



图6 论文最重要的经验规律：Agent 越强，AdaCoM 越倾向保留高保真 Raw Context

AdaCoM 为四个 Agent 学到的平均 post-management context length 分别约为 DeepSeek 1.9K、Kimi 3.4K、Qwen 5.2K、GLM 7.0K tokens，而且这个顺序与 Vanilla ReAct 能力排序一致。

9.1 两种涌现策略

- Eager distillation：DeepSeek / Kimi 几乎每轮压缩，持续维持短而明确的状态。
- Tiered management：Qwen / GLM 先保留原始 tool results，让 context 增长，再偶尔批量压缩。

9.2 Know-why

“更多原始上下文 = 更好”与“越短越好”都不成立。原始历史保留细节、证据和失败路径，但超过某个 Agent 的 effective context length 后，会降低 reasoning reliability。压缩提高可靠性，却可能丢失细节。因此最优策略是对具体 Agent 找到 Fidelity 与 Reliability 的 Pareto 平衡。

这对 Context Engineering 是非常重要的范式变化：Context budget 不应只按模型宣称的 128K/256K window 配置，而应按 Agent 在任务中的有效处理能力配置。

10. Cross-Agent Transfer : Context Manager 能否用

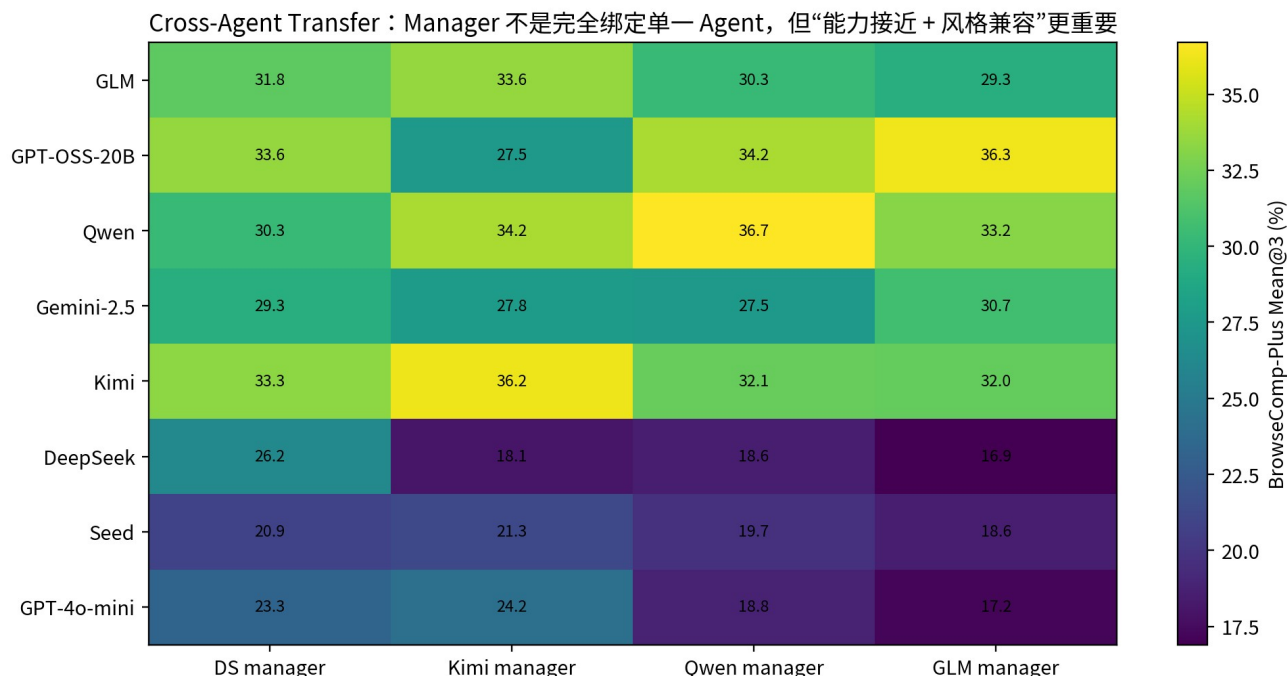


图 7 Cross-Agent Transfer: 同一个 Manager 在多个未训练目标 Agent 上仍可提升, 但风格兼容性会影响效果

论文把 4 个已训练 Manager (DeepSeek/Kimi/Qwen/GLM) 交叉部署到 8 个目标 Agent, 包括 4 个 source agents 与 4 个 held-out agents。28 个 cross-agent pair 中有 23 个获得正收益; 加上 4 个 self-trained pair, 32 个配置平均相对提升 25.0%, 纯 cross-agent 平均 +22.1%。

最大 cross-agent 增益出现在 Kimi 使用 DeepSeek-trained manager: +79.6%, 接近 Kimi self-trained manager 的 +95.0%。这说明 Context Policy 有成为“可复用系统组件”的潜力。

10.1 但 capability proximity 不是全部

- DeepSeek 与 Kimi baseline 能力接近, 但 DeepSeek 更偏好 concise key findings + document IDs; Kimi manager 会保留更详细的 query / doc history, 迁移给 DeepSeek 反而可能引入无用负担。
- GLM 对 aggressive compression 敏感, 因此较弱 Agent 学出的 Manager 迁移给 GLM 效果不佳。
- Gemini 对 Qwen manager 的 report-style working memory 利用不好, 说明“记忆文风/结构”也是 compatibility 变量。

工程建议: 不要为每个模型都训练独立 Context Manager; 先按照 vanilla agent capability 分组训练少量代表 Manager, 再用小规模 A/B eval 在组内选择 style-compatible manager。

11. 数据构造与训练工程

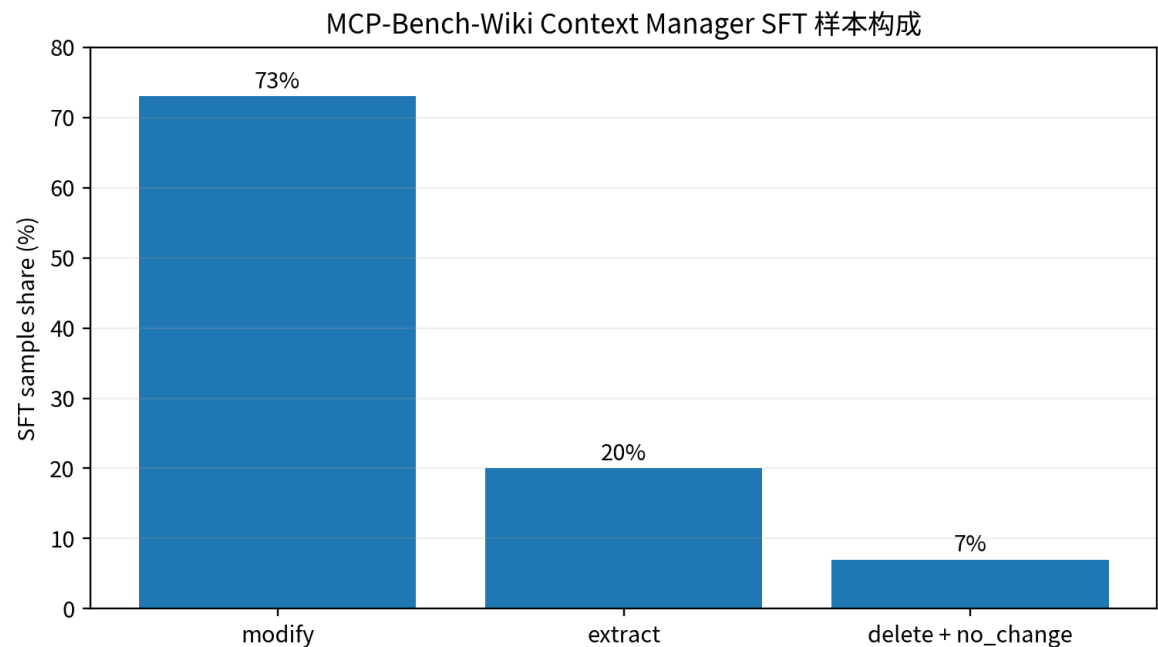


图 8 MCP-Bench-Wiki SFT 数据并不是“全摘要”，而是以通用 modify 为主

MCP-Bench-Wiki 的训练数据更接近真实 Deep Research。作者从约 3,500 个训练侧多模型任务生成中筛到 2,517 个干净任务，再得到 4,740 个 per-invocation SFT samples、1,000 个 task-level RL tasks，以及独立的 150 task test set。

数据层	筛选/规模	重要设计
Raw generation	约 3,500 train / 1,000 test	训练/测试来自独立上游 run
Upstream filter	2,517 train / 879 test	完成状态、答案长度、task fulfillment、embedding dedup、LLM quality judge
SFT	4,740 invocations	约 73% modify、20% extract、7% delete/no_change
RL	1,000 tasks	task fulfillment >=3; 10-39 reasoning rounds
Test	150 tasks	30-150 tool calls; reference >500 chars; 额外质量过滤

这套数据构造的 know-how 是：Context Manager 训练不能只收集“好摘要”，而要覆盖完整 edit distribution；同时要在 task-level 严格隔离训练与测试，避免把具体 task state 模板记住。

12. 与 ReSum / LongSeeker / ACM / ACE / PACE 的关系

2026 Context Engineering：关键问题从“怎么压缩”变成“谁来管理、如何适配 Agent、能否恢复”

方法	谁管理 Context	动作空间	是否需训练主 Agent	可恢复性/外存
ReSum	外部 Summary Tool	整体 summarization	否；但 ReSum-GRPO 可适配 Agent	低
LongSeeker	主 Agent	Skip/Compress/Rollback/Snippet/Delete	是 (SFT Agent)	低
AdaCoM	外部 Manager LLM	任意 message rewrite/merge/delete	否；训练 Manager	低
ACM	主 Agent + 外部 Memory	compress/offload/retrieve	后训练 Agent	高
ACE	外部 Elasticizer	raw / abstract / drop 每步重编排	否 (plug-and-play)	高：raw 保存在维护层
PACE	预测式 Context Extractor	按 next-step relevance 调粒度	独立框架	中

图 9 2026 Context Management 方法的核心分叉：管理主体、动作空间、训练对象、可恢复性

12.1 AdaCoM vs ReSum

两者都可把管理能力放在主 Agent 外部，但 ReSum 的 operation 固定为 summarization；AdaCoM 允许自由修改 message sequence，并通过 RL 针对具体 Agent 学习策略。ReSum 的优势是简单、plug-and-play；AdaCoM 的优势是 agent compatibility 与可迁移性。

12.2 AdaCoM vs LongSeeker / Context-ReAct

LongSeeker 把 Context Action 与 Search Action 合并在主 Agent policy 中，动作语义明确且粒度细，但需要训练主 Agent。AdaCoM 反过来彻底 decouple：主 Agent 冻结，外部 Manager 学习任意 message edit。这使 AdaCoM 更适合闭源/多模型平台。

12.3 AdaCoM vs ACM

ACM（2026-07）把“被压缩/移除的信息可恢复”作为核心：内容 offload 到 external memory，Agent 可以按需 retrieve。AdaCoM 的修改更灵活，但被 delete/重写的 working-context 信息本身没有论文级别的 lossless cold-store recovery 机制。

12.4 ACE / PACE：下一步的两个方向

- ACE：保存 raw + abstract 的 lossless maintenance layer，每个 decision step 决定 raw / abstract / drop，强调可逆。
- PACE：把 Context 选择重构成 Next Step Prediction，根据下一动作相关性动态调历史粒度，并在 ultra-long horizon 中扩展到数千步。

13. 生产级 Deep Research 的可迁移 Know-how

13.1 把 Context Manager 做成独立 Control Plane

推荐把生产 Research Agent 拆成 Search Policy、Context Policy、Evidence Store、Artifact State、Verifier。主模型负责研究决策；Context Manager 负责每次调用前构造最适合主模型的 working set。

- 优点 1：更换主 Agent 时不必重写整个 context pipeline。
- 优点 2：闭源模型也能适配，只需拿到其 tool loop 输入/输出。
- 优点 3：Context Manager 可以用更便宜的小模型运行，并做 capability-based routing。

13.2 Message Edit IR 应保存 provenance

论文的 JSON edit schema 已经是很好的最小 IR，但生产环境应再加入 source message IDs、evidence refs、edit type、timestamp、confidence、reversible pointer。这样 rewrite 后仍能追溯原始证据。

13.3 Working Context ≠ Full Trace ≠ Evidence Store

层	允许压缩/删除?	用途
Working Context	允许	供当前 LLM 推理，追求高信噪比
Full Trace	不允许永久删除	审计、debug、训练 replay
Evidence Store	不可丢失原始证据	claim grounding、citation verification、按需恢复

13.4 Agent Compatibility Profile

可以给每个 Agent 维护一个 context profile：effective context range、偏好的 memory style、summary density、raw evidence tolerance、tool-loop repetition sensitivity。AdaCoM 的 Fidelity-Reliability Trade-off 提供了这种 profiling 的实验依据。

14. 局限、风险与下一代架构

- 不可逆编辑风险：Manager rewrite/delete 后，若没有独立 raw trace/evidence store，后续 Agent 可能无法恢复关键细节。
- Manager observability：论文优化最终任务 reward，但生产系统还应显式度量 evidence retention、constraint recall、citation coverage、context churn。
- KV cache 成本：频繁重写历史会破坏 prefix/KV cache。论文也指出更强 Agent 可采用 tiered management，靠近 effective limit 再压缩，以减少 cache-breaking operations。
- 训练成本：当前结果为每个 source Agent 单独训练 Manager；虽然 cross-agent transfer 有效，但选择最兼容 Manager 仍需要 eval。
- 任务范围：论文主要覆盖 knowledge-intensive search / deep research；代码 Agent、GUI Agent、embodied Agent 是否同样成立需要进一步验证。

14.1 推荐的 Next-Gen Architecture

更完整的生产方案应结合 AdaCoM + ACM/ACE：外部 RL-trained Context Policy 决定 active view，但所有被压缩内容进入 lossless external memory；每个 summary 保留 evidence handles，Agent 或 Context Manager 可在需要时 rehydrate 原文。

可以抽象为：ManagedContext_t = Policy(AgentProfile, TaskState, RecentTrace, MemoryIndex, TokenBudget)，其中 Policy 的 action 不只是 rewrite/delete，还包含 pin_raw、summarize、offload、retrieve、reorder、merge、no_change。

15. 结论

AdaCoM 最值得记住的不是某个具体分数，而是一个架构原则：Context Management 与 Reasoning/Search Policy 可以解耦，而且“最佳上下文”是 Agent-dependent 的。

从工程角度看，Deep Research 的竞争正在从“谁有更大 Context Window”转向“谁能在每一步构造更高价值的 Working Context”。AdaCoM 提供了一条特别适合多模型平台的路线：冻结主 Agent，用小型外部 Manager + RL 学习 agent-compatible context transformations。

最终可沉淀为：

$$\text{Long-Horizon Agent Reliability} \approx \text{Task Policy} \times \text{Agent-Compatible Context Policy} \times \text{Lossless Evidence Memory} \times \text{Verifier} \times \text{Budget Controller}$$

Know-why：上下文不是越多越好，也不是越短越好；不同 Agent 存在不同 effective context regime。Know-how：把 Context Manager 做成独立可训练控制层，用任务结果 + 过程信号优化，并将 Working Context 与原始 Evidence/Trace 解耦。

参考资料

- [1] Lu Yi, Runlin Lei, Liuyi Yao, et al. Learning Agent-Compatible Context Management for Long-Horizon Tasks. arXiv:2605.30785, 2026-05-29.
- [2] Yijun Lu, Rui Ye, Yuwen Du, et al. LongSeeker: Elastic Context Orchestration for Long-Horizon Search Agents. arXiv:2605.05191, 2026.
- [3] Xixi Wu, Kuan Li, Yida Zhao, et al. ReSum: Unlocking Long-Horizon Search Intelligence via Context Summarization. arXiv:2509.13313, 2025.
- [4] Xiaochuan Li, Ryan Ming, Meng Chu, et al. ACM: Agentic Context Management for Long Horizon Tasks. arXiv:2607.23809, 2026.
- [5] Ning Liao, Zihao Long, Xiaoxing Wang, et al. ACE: Pluggable Adaptive Context Elasticizer across Agents. arXiv:2606.31564, 2026.
- [6] Lei Wei, Xiao Peng, et al. PACE: Predictive Adaptive Context Extraction for Long-Horizon LLM Agents. ACL 2026 Long Papers.
- [7] AdaCoM paper experimental details: BrowseComp-Plus, MCP-Bench-Wiki, Trinity-RFT, Qwen3-4B-Instruct manager, process-reward ablations and cross-agent transfer analyses are taken from the paper and appendices.
- [8] Code availability note (2026-08-13): the arXiv paper page does not expose an official code repository link, and an exact-title GitHub repository search did not return a corresponding official repository. Therefore this report does not invent source-code-level AdaCoM implementation details; implementation discussion is limited to the paper-provided protocol and training setup.